

VIRELYON

AI WORKFORCE PLATFORM

CAHIER DES CHARGES FONCTIONNEL UNIFIE

Les 3 agents autonomes de la plateforme

ARES - APEX - AURA

Version 3.0 - Document unique de reference technique

Stack backend : Python / FastAPI

Public : Dev IA - Dev Big Data - Dev Fullstack - Dev Frontend

Document interne - usage exclusif de l'equipe technique VIRELYON

Sommaire

- 0. Comment lire ce document
- 1. Vision, positionnement et cible produit
- 2. Architecture technique globale (Backend Python / FastAPI + Supabase + Claude + n8n)
- 3. Modele de donnees unifie (Postgres)
- 4. Agent ARES - Prospection commerciale
- 5. Agent APEX - Support client
- 6. Agent AURA - Reporting et Analytics
- 7. Prompts systeme - contrat d'entree/sortie
- 8. Integrations tierces (par agent)
- 9. Parcours transversaux inter-agents
- 10. Garde-fous non negociables (consolides)
- 11. Exigences non-fonctionnelles
- 12. Feuille de route technique
- 13. Glossaire

0. Comment lire ce document

Ce document est la reference technique unique pour les trois agents autonomes de VIRELYON : ARES (prospection), APEX (support client) et AURA (reporting et pilotage). Il remplace les documents separes precedents et corrige les incoherences d'architecture entre les versions anterieures.

0.1 Public cible

- Developpeur IA - conçoit les prompts systeme, les schemas Pydantic, ajuste les seuils de confiance et d'escalade.
- Developpeur Big Data - construit les pipelines de sourcing (ARES), l'indexation vectorielle pgvector (APEX), les requetes SQL de calcul (AURA).
- Developpeur Fullstack - construit le backend Python (FastAPI) qui porte toute la logique des trois agents.
- Developpeur Frontend - consomme les endpoints REST exposes par le backend Python (FastAPI), construit les ecrans de configuration et de dashboard.

0.2 Structure du document

Chaque agent (Sections 4, 5, 6) est traite avec la meme grille de lecture, module par module, en 5 blocs : Objectif - Logique et comportement - Tableau Declencheur/Decision/Resultat - Etats et limites - Donnees et integrations. Un developpeur qui rejoint le projet doit pouvoir lire ce seul document et savoir exactement quoi construire, dans quel ordre, avec quelles regles non negociables.

0.3 Conventions typographiques

- Encadre rouge = garde-fou non negociable, critere de recette avant mise en production.
- Encadre violet = decision d'architecture actee (non ouverte au debat).
- Encadre vert = information contextuelle ou rappel utile.
- Les tableaux Declencheur/Decision/Resultat sont testables : une ligne = un cas verifiable en recette.

1. Vision, positionnement et cible produit

1.1 Les trois agents en une phrase chacun

- ARES - le commercial qui ne dort jamais : il cherche, qualifie, contacte et relance les prospects a la place de l'entreprise, sur donnees publiques (Google Maps, LinkedIn, sites web, annuaires B2B type Hunter ou Apify), et transfere le lead qualifie a l'equipe commerciale.
- APEX - l'employe du support qui repond en moins de 3 secondes : il traite les questions recurrentes des clients finaux a partir de la base de connaissances fournie par l'entreprise, et transfere a un humain des que la situation devient sensible ou hors perimetre.
- AURA - l'analyste qui ne dort jamais : il consolide les donnees reelles de l'entreprise (CRM, facturation) et la performance d'ARES et APEX, et produit des rapports clairs sans jamais inventer un chiffre.

1.2 Principe fondateur commun : augmenter, jamais remplacer

POSITIONNEMENT NON NEGOCIABLE

Aucun texte genere par ARES, APEX ou AURA ne doit suggerer qu'il remplace un humain. Le positionnement valide est augmenter. Ce principe conditionne chaque prompt systeme et chaque texte genere par les trois agents, sans exception. Il est verifie par un test de recette dedie avant toute mise en production.

1.3 Cible produit

- Entreprise de service B2B uniquement (marketing, conseil, communication, RH, developpement logiciel, pharmacie).
- Aucune fourchette d'effectif imposee - la taille du client n'est pas un filtre.
- Hors perimetre explicite : hotelier, e-commerce, retail physique.

1.4 Positionnement Orange

Orange Business est le premier canal de distribution de VIRELYON, pas une dependance technique. Les trois agents fonctionnent integralement pour toute entreprise de service B2B dans le monde, cliente d'Orange ou non. Aucune integration Orange n'est un prerequisite bloquant a l'activation d'un agent.

Deploiement geographique : Madagascar puis Afrique francophone (via Orange) puis France / Belgique / Suisse.

1.5 Contraintes de conception permanentes

Toute decision technique de ce document a ete filtree par quatre criteres, dans cet ordre de priorite :

- Demontrable en moins de 5 minutes devant un jury, module par module.
- Faisable dans le calendrier du programme d'incubation en cours (11 a 12 semaines).
- Deployable multi-pays sans reprise de code : multilinguisme et isolation par workspace natifs, pas ajoutes apres coup.
- Zero dette technique qui empecherait l'evolution conjointe des trois agents dans la meme architecture.

2. Architecture technique globale

DECISION D'ARCHITECTURE ACTEE

Un backend Python FastAPI unique porte toute la logique metier des trois agents : garde-fous, decisions, appels Claude API, calculs, ecriture des evenements. n8n est conserve mais recentre sur son role de connecteur no-code vers des outils tiers (email, Slack, WhatsApp Business API, sourcing externe, export webhook). Aucune logique de decision critique ne reside plus dans n8n.

2.1 Composants

Composant	Role	Outil / techno
Backend applicatif	Porteur exclusif de toute la logique metier des trois agents.	Python / FastAPI
Base de donnees	Isolation stricte par workspace (Row Level Security), memoire vectorielle.	Supabase Postgres + extension pgvector
Intelligence artificielle	Seul moteur de raisonnement. Tous les appels partent du backend Python (FastAPI).	Claude API (Anthropic)
Automatisation externe	Connecteurs no-code vers outils tiers (email, Slack, WhatsApp, CRM). Aucune decision.	n8n
Migrations de schema	Versioning des evolutions Postgres, revues comme du code.	Alembic
Frontend	Consomme exclusivement les endpoints REST du backend Python (FastAPI). Aucun acces direct a Supabase.	Hors perimetre de ce CDCF

2.2 Pourquoi Python et FastAPI (justification)

- Une seule pile pour toute la logique agent (Python), coherente avec la stack Big Data / ML deja utilisee - pas de rupture de competence entre equipes.

- Typage et validation de schema natifs (Pydantic) : les sorties structurees des appels Claude sont verifiables automatiquement, pas seulement documentees.
- Documentation OpenAPI generee automatiquement : le contrat entre backend et frontend n'a jamais a etre redige a la main.
- n8n reste excellent pour brancher rapidement un canal tiers, mais devient inadapte des qu'on lui confie une decision critique (garde-fou anti-hallucination, catalogue ferme d'actions). Cette logique doit etre du code versionne, teste, revu.

2.3 Repartition des responsabilites entre developpeurs

Role	Responsabilite principale	Livrable attendu
Dev IA	Prompts systeme versionnes (Section 7), schemas Pydantic de sortie structuree, seuils de confiance et d'escalade.	Bibliotheque de prompts + modeles Pydantic dans app/*/prompts.py et app/*/schemas.py.
Dev Big Data	Pipelines de sourcing/ingestion (ARES), indexation vectorielle pgvector (APEX), requetes SQL de calcul (AURA).	Connecteurs d'ingestion testes sur donnees reelles, requetes SQL optimisees.
Dev Fullstack (Backend Python / FastAPI)	Endpoints REST, orchestration Claude, garde-fous, ecriture des evenements, authentication, isolation par workspace.	Service backend Python (FastAPI) deploye, OpenAPI documente automatiquement.
Dev Frontend	Ecrans de configuration et de dashboard des trois agents. Consomme uniquement les endpoints du backend Python (FastAPI).	Interfaces connectees a l'API, aucun acces direct a Supabase.

2.4 Arborescence du backend Python (FastAPI)

Structure par domaine metier plutot que par type technique, pour que chaque agent reste un module isole et testable independamment :

- app/core/ - configuration, securite, dependances partagees (authentication, workspace_id).
- app/db/ - session Postgres (SQLAlchemy async / asyncpg), modeles ORM.
- app/claude/ - client Claude API unique, wrapper d'appel, gestion des retries.
- app/ares/ - router.py (endpoints REST), service.py (logique de decision, catalogue d'outils, sourcing), schemas.py (Pydantic), prompts.py (prompts versionnes).
- app/apex/ - router.py, service.py (RAG, escalade, generation de reponse), schemas.py, prompts.py.
- app/aura/ - router.py, service.py (consolidation, calcul d'indicateurs, garde-fou anti-invention), schemas.py, prompts.py.
- app/events/ - ecriture des tables d'observabilite (lead_events, conversation_events, aura_events).
- alembic/ - migrations de schema Postgres.
- tests/ares/, tests/apex/, tests/aura/ - tests unitaires et de recette par agent.

2.5 Flux de donnees (schema des couches)

Requete entrante (canal ou frontend) -> Endpoint FastAPI de l'agent concerne -> Service metier (applique la logique et les garde-fous) -> Appel Claude API si necessaire (raisonnement, jamais calcul brut) -> Lecture/ecriture Postgres via SQLAlchemy -> Ecriture d'un evenement d'observabilite -> Reponse structuree renvoyee a l'appelant.

PRINCIPE NON NEGOCIABLE - CALCUL

Claude ne calcule jamais un agregat numerique lui-meme. Tout calcul passe par du SQL execute depuis le service FastAPI. Le modele redige et decide de la classification, il ne compte pas. Regle identique pour les trois agents.

Flux de donnees - architecture FastAPI unique

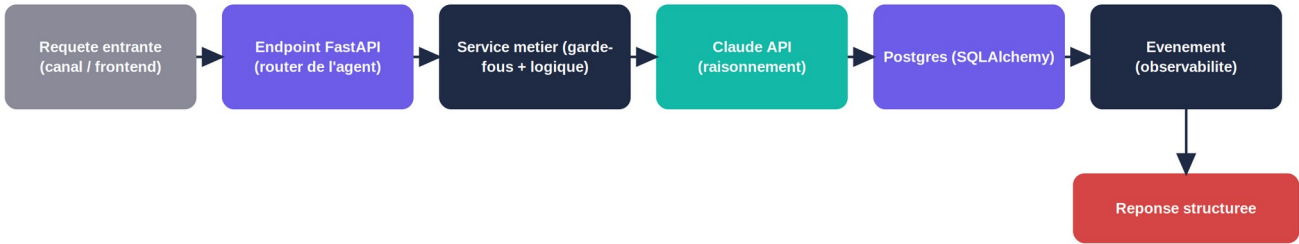


Figure 1 - flux de donnees, de la requete entrante a la reponse structuree, identique pour les trois agents.

3. Modele de donnees unifie (Supabase Postgres)

3.1 Conventions communes

- Chaque table metier porte un champ workspace_id (uuid), avec une policy RLS Postgres qui filtre systematiquement sur ce champ.
- Chaque table d'evenements (lead_events, conversation_events, aura_events) suit le meme schema : id, workspace_id, type_evenement, horodatage, justification, detail (jsonb) - pour permettre a AURA une agregation croisee sans transformation.
- Les migrations de schema sont geres par Alembic, versionnees dans le depot backend - plus de modification manuelle du schema hors migration.
- Aucun secret d'integration n'est stocke en base : les cles API (Claude, HubSpot, Stripe, Orange) vivent dans les variables d'environnement du service FastAPI.

3.2 Tables ARES

workspace_icp_config - partagee, propriete du workspace

Champ	Type	Description
workspace_id	uuid	Cle du workspace propriétaire - source unique de verite pour tout critere de ciblage.
secteurs	text[]	Secteurs d'activite cibles, librement definis par le client - aucune liste fermee imposee.
zones_geographiques	text[]	Zones cibles par le client, optionnel.
mots_cles	text[]	Mots-cles / signaux recherches (technologies utilisees, intitules de poste, etc.).
effectif_min / effectif_max	int, nullable	Optionnels et non pre-remplis. Aucune plage fixe par default.
updated_at / updated_by	timestamp / uuid	Tracabilite des modifications de ciblage.

leads_raw - alimentee par le pipeline Big Data

Champ	Type	Description
id / workspace_id	uuid	Identifiants.
source	text	Origine : sourcing_api / import_csv / formulaire_inbound.
donnees_brutes	jsonb	Profil brut de l'entreprise ou du contact avant qualification.

Champ	Type	Description
hash_dedup	text	Empreinte de deduplication (email / domaine / nom d'entreprise normalises).
ingested_at	timestamp	Horodatage d'ingestion, utilise pour le score de fraicheur.

leads - pipeline actif, consomme par l'interface

Champ	Type	Description
id / workspace_id	uuid	Identifiants.
qualification_status	enum	qualifie / rejete / a_valider.
qualification_reason	text	Justification courte retournee par Claude - tracabilite obligatoire.
score	numeric	Score composite de priorisation (0-100). Pilote aussi le palier de relance.
score_breakdown	jsonb	Detail du calcul de score, pour transparence et debogage.
relances_effectuees	int	Compteur de relances deja envoyees, compare au plafond du palier avant chaque nouvelle tentative.
statut_pipeline	enum	Identifie / Contacte / Repondu / Qualifie / Transmis au commercial.
canal_preferre	text	Canal actif retenu pour ce lead.

Autres tables ARES

Table	Role
sequences / sequence_steps	Une sequence par configuration d'ARES active. Champs : step_order, canal (email / linkedin / sms / whatsapp), delai_jours, condition_arret (ex. reponse recue, toujours prioritaire sur la poursuite).
messages	Historique des messages generes : canal, contenu_genere (conserve integralement pour audit), statut_envoi (en_attente / envoye / echec / bloque_suppression_list), genere_par_prompt_version.
suppression_list	Conformite RGPD / opt-out. Champs : contact_identifiant, portee (workspace / global), motif, ajoute_le. Invulnerable : un blocage reste actif meme apres changement de configuration.
agent_config_ares	Reglages d'execution : statut (Configuration incomplete / Inactif / Actif), autonomy_level (supervision / autonome), canaux_actifs, quotas_par_canal, paliers_relance_score.
embeddings	Memoire vectorielle (pgvector). Champs : lead_id / event_id, vecteur, usage (dedup_semantique / reprise_contexte).
lead_events	Journal d'observabilite d'ARES, consomme en lecture seule par AURA. Schema commun (Section 3.1).

3.3 Tables APEX

Table	Role
knowledge_base_documents	Documents et FAQ fournis par le client (PDF, DOCX, TXT, saisie directe).
apex_knowledge_chunks	Fragments vectorises (pgvector) de la base de connaissances, unite de recherche semantique (RAG).
conversations	Pipeline actif d'une conversation : canal, contact, statut, sentiment detecte.
messages_apex	Historique complet des echanges d'une conversation, conserve pour audit.
regles_escalade	Configuration des declencheurs d'escalade par workspace : mots-cles, seuils de sentiment, nombre d'echanges, hors_perimetre_connaissance.

Table	Role
agent_config_apex	Reglages d'execution : portee (scope autorise), niveau_autonomie (Moyen / Eleve), ton de voix, canaux actifs, quotas par canal, statut de l'agent.
feedback_satisfaction	Note de satisfaction optionnelle collectee en fin de conversation.
conversation_events	Journal d'observabilite, consomme en lecture seule par AURA. Schema commun (Section 3.1).

3.4 Tables AURA

Table	Role
report_config	5 reglages saisis par le client : objectif de suivi, indicateurs actifs, frequence, format, canaux, ton de voix.
data_source_connections	Etat des connexions aux sources externes (CRM, facturation) et horodatage de derniere synchronisation - base du garde-fou de fraicheur.
indicateurs_catalogue	Referentiel des indicateurs disponibles, hors perimetre workspace.
indicateurs_calcules	Valeurs calculees par periode et par indicateur, avec statut (calcule / donnee_manquante / erreur) - jamais uniquement la valeur finale.
rapports_generes	Historique des rapports generes, texte conserve integralement pour audit.
donnees_manquantes_signalees	Journal du garde-fou anti-invention : chaque indicateur non calculable est trace, jamais masque.
aura_events	Journal d'observabilite propre a AURA - indispensable a l'audit interne.

4. Agent ARES - Prospection commerciale

EN UNE PHRASE

ARES est le commercial qui ne dort jamais : il cherche des prospects sur donnees publiques, les qualifie selon l'ICP defini par le client, engage la conversation sur le canal choisi, relance selon le palier de score, et transfere le lead qualifie a l'equipe commerciale - jamais en remplacement, toujours en renfort.

Toute la logique de decision d'ARES decrite ci-dessous est implementee dans le backend Python (FastAPI), module app/ares/ - jamais dans n8n (Section 2).

4.1 Ce qu'ARES fait, ce qu'ARES ne fait jamais

Ce qu'ARES fait

- Recherche des prospects sur des sources publiques : Google Maps (entreprises par secteur/ville), LinkedIn (decideurs), sites web publics, annuaires B2B professionnels type Hunter (verification d'emails).
- Elimine les doublons et evalue la pertinence de chaque lead par rapport a l'ICP du client (lecture seule).
- Engage la conversation (email, LinkedIn, SMS, WhatsApp, Slack selon canaux actifs), adapte le message au ton de voix configure.
- Relance selon un palier de score : plus le lead est qualifie, plus l'effort de relance est soutenu (Section 4.5).
- Transfere le lead qualifie a l'equipe commerciale et l'ajoute automatiquement au CRM du client (HubSpot, Salesforce).

Ce qu'ARES ne fait jamais

- Aucun acces a des donnees privees ou confidentielles (mots de passe, bases piratees, infos internes d'entreprise).
- Aucune action hors du catalogue ferme (Section 4.3) - ARES ne peut jamais improviser une action non prevue.
- Aucun message qui suggere qu'ARES remplace un commercial humain.
- Aucune ecriture dans la table d'evenements sans horodatage ni justification.

4.2 Les 5 reglages cote client

Le client configure ARES en 5 minutes via l'ecran Agent Builder. Aucune ligne de code. Cette section decrit ce que le frontend doit exposer et ce que le backend FastAPI doit accepter.

Reglage	Ce que le client saisit	Champ backend (agent_config_ares / workspace_icp_config)
1. Objectif principal	Une phrase libre (ex. Generer des leads qualifies dans le secteur services).	workspace_icp_config.objectif_principal (text)
2. ICP (profil client cible)	Secteurs, zones, mots-cles, effectif optionnel.	workspace_icp_config (voir Section 3.2)
3. Formation d'ARES	Upload de scripts d'approche, exemples d'emails qui ont marche, docs produit, liste de contacts existants (drag and drop PDF / Word / CSV).	Documents indexes dans knowledge_base_documents (partage APEX) + hash de deduplication pour la liste de contacts.
4. Canaux	Cases a cocher : Email, WhatsApp, LinkedIn, SMS, Slack.	agent_config_ares.canaux_actifs (jsonb)
5. Ton de voix	Pro / Chaleureux / Direct / Creatif.	agent_config_ares.ton_de_voix (enum)

Test avant lancement : le client doit pouvoir saisir une phrase comme s'il etait un prospect et voir en direct ce qu'ARES repondrait - avec sa formation et son ICP charges - avant de le lancer sur de vrais contacts.

4.3 Decision d'architecture : agent hybride a catalogue ferme

Deux approches ecartees. Un script fige (regles si/alors) est simple mais casse des qu'un prospect sort du chemin prevu. Un agent totalement libre expose a des hallucinations et boucles imprevisibles. Decision retenue : orchestrateur LLM avec catalogue ferme d'outils, porte par app/ares/service.py.

Claude recoit a chaque etape le contexte du lead (profil, historique, statut CRM) et choisit librement la prochaine action, mais uniquement parmi cette liste fermee et non extensible :

- envoyer_email
- relancer
- qualifier_besoin
- proposer_rdv
- escalader_vers_humain
- marquer_non_qualifie

GARDE-FOU - CATALOGUE FERME

ARES ne choisit jamais une action hors de ce catalogue. Toute nouvelle capacite doit d'abord etre ajoutee au catalogue dans app/ares/service.py - jamais laissee a l'improvisation du modele. Chaque decision et chaque action executee est journalisee dans lead_events.

4.4 Sourcing et ingestion des prospects

Objectif : alimenter ARES en prospects bruts pertinents, sans jugement de qualite a ce stade - le jugement reste au module Qualification ICP (Section 4.5). Chaque source est un connecteur externe (relais n8n ou webhook direct) qui alimente l'endpoint FastAPI d'ingestion.

Source	Role	Zone
Google Maps	Sourcing d'entreprises par secteur / ville - B2B local et geolocalise.	Madagascar + international
LinkedIn (public)	Sourcing de decideurs et enrichissement de profil.	International
Hunter	Recherche et verification d'emails professionnels valides.	International
Apify	Scraping structure (annuaires, sites web, profils publics).	International
Scraping web generique	Lecture d'informations publiques pour enrichir le profil avant qualification.	Toutes zones
Import CSV manuel	Import direct d'une liste existante par le client.	Toutes zones
Formulaire inbound (webhook)	Captation directe d'un lead entrant.	Toutes zones

- Regle systematique : chaque lead brut, quelle que soit sa source, est deduplique (email / domaine / nom d'entreprise normalises) avant toute ecriture en base.
- Aucune source n'est un prerequis : ARES fonctionne meme en import CSV seul.
- Volume anormalement eleve sur une execution : seuil d'alerte et pause automatique en attente de validation humaine.
- Execution planifiee par default (quotidienne), avec declenchement manuel possible depuis le frontend.

Endpoints FastAPI de reference : *POST /ares/{workspace_id}/sources/{type_source}/ingest*, *POST /ares/{workspace_id}/leads/import-csv*

4.5 Qualification ICP (lecture seule)

Objectif

Determiner si un lead brut correspond au profil defini par le client, en lecture seule sur workspace_icp_config. ARES ne definit et ne stocke jamais son propre filtre ICP.

Logique et comportement

- Chaque critere de workspace_icp_config est optionnel et defini par le client - aucune valeur par default de plage d'effectif fixe. La cible reste toute entreprise de service B2B, sans fourchette de taille.
- Claude evalue chaque lead brut contre les criteres actifs et retourne un statut structure (qualifie / rejete) accompagne d'une justification courte, pour tracabilite.
- Un lead rejete n'est jamais supprime : il reste archive avec son motif, pour audit et amelioration future du ciblage.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Nouveau lead dans leads_raw	Appel Claude avec workspace_icp_config + profil du lead	Statut qualifie -> visible dans le pipeline, ou rejete -> archive	Sortie JSON Pydantic obligatoire - jamais de texte libre
Confiance insuffisante du modele	Ne pas trancher automatiquement	Statut a_valider, file dediee pour revue humaine	Jamais un rejet silencieux en cas d'ambiguite
Modification de workspace_icp_config par le client	Aucune reevaluation retroactive automatique	Bouton Reevaluer les leads en attente disponible a la demande	Reevaluation de masse = action volontaire, pour maitriser le cout d'appels API

Etats et limites

- workspace_icp_config vide : aucun lead ne peut etre qualifie tant que le client n'a pas renseigne son profil cible.
- Aucun critere par default n'est jamais applique silencieusement - un ICP vide bloque la qualification.

Donnees et integrations

- Lecture : workspace_icp_config (hors perimetre d'ecriture d'ARES).
- Ecriture : leads.qualification_status, leads.qualification_reason.

Endpoints FastAPI de reference

- POST /ares/{workspace_id}/leads/{lead_id}/qualify

4.6 Scoring, priorisation et paliers de relance

Objectif

Ordonner les leads qualifies pour maximiser l'usage du temps de contact disponible, et piloter directement le comportement du module Sequencage (Section 4.7).

Logique et comportement

- Score composite base sur la fraicheur du signal, la completude des donnees, la force de correspondance avec l'ICP, et l'historique d'engagement en cas de recurrence.
- Ponderation configurable par workspace, valeurs par default fournies mais jamais figees en dur dans le code.
- Le score pilote le palier de relance : un lead a faible score ne mobilise pas la meme energie de contact qu'un lead a score eleve.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Lead qualifie	Calcul du score composite (SQL depuis FastAPI, jamais par Claude)	Insertion / mise a jour dans la vue priorisee du pipeline	score_breakdown stocke pour transparence
Nouvelle information sur un lead existant (ex. reponse recue)	Recalcul du score	Repositionnement dans la file de priorite	Declenche par le module Classification (Section 4.9)

Etats et limites

- Signaux insuffisants pour scorer : score neutre par défaut + indicateur données incomplètes affiche, jamais un score trompeur.

Données et intégrations

- Table leads (score, score_breakdown, relances_effectuees).
- Table agent_config_ares (paliers_relance_score).

4.6.1 Paliers de score et cadence de relance

Palier de score	Interpretation	Relances max	Cadence
superieur ou egal a 95%	Correspondance quasi-parfaite avec l'ICP et signal fort	5	J0 -> J+3 -> J+7 -> J+12 -> J+18 -> J+25
90 - 94%	Tres forte correspondance	4	J0 -> J+3 -> J+8 -> J+15 -> J+22
70 - 89%	Correspondance correcte (cas standard)	3	J0 -> J+4 -> J+10 -> J+18
inferieur a 70%	Correspondance faible	1	J0 -> J+7, puis archivage automatique

Avant chaque relance programmee, le moteur de sequencage compare leads.relances_effectuees au plafond du palier dans agent_config_ares.paliers_relance_score, et n'exécute la relance que si le plafond n'est pas atteint. Un recalcul de score en cours de sequence peut faire changer un lead de palier : le nouveau palier s'applique uniquement aux relances futures, jamais retroactivement.

4.7 Sequencage multicanal

Objectif

Executer une cadence de contact multi-etapes et multi-canaux sans intervention manuelle, en respectant les quotas de chaque canal et les paliers de relance (Section 4.6.1).

Logique et comportement

- Verification des quotas et limites du canal avant tout envoi, et respect des fenetres horaires locales du prospect.
- Avant toute relance, verification du palier de score du lead : relances deja effectuees comparees au plafond autorise.
- Des qu'une reponse est detectee, la sequence est interrompue immediatement pour ce lead - jamais de message automatique apres une reponse humaine tant que la classification n'a pas eu lieu.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Etape J0/J+n programmee	Verification du quota du canal	Envoi, ou mise en file d'attente si quota atteint	Un connecteur n8n par canal, orchestre par app/ares/service.py
Absence de reponse apres une etape	Verifier relances_effectuees < plafond du palier	Relance programmee selon la cadence, ou arret definitif (archivage)	Le nombre de relances depend du score, jamais d'une regle fixe uniforme
Reponse detectee sur un canal	Interruption immediate de la sequence pour ce lead	Transfert vers le module Classification (Section 4.9)	Priorite absolue sur toute autre regle de cadence
Echec technique d'envoi (ex. email rejete)	Marquer le canal invalide pour ce lead	Bascule sur canal alternatif si configure, sinon pause + alerte	Jamais de nouvelle tentative infinie silencieuse

Etats et limites

- Canal non connecte : etape ignoree avec journalisation, jamais un echec silencieux.
- Quota epuise : mise en file d'attente, reprise automatique a la reouverture du quota.

- Plafond de relance atteint : lead archive avec motif, jamais supprime.

Donnees et integrations

- Tables sequences, sequence_steps, messages ; lecture de leads.score, leads.relances_effectuees et agent_config_ares.paliers_relance_score.

4.8 Generation de messages personnalisés

Objectif

Produire, a chaque etape, un message pertinent et non generique par lead, dans le respect strict du positionnement produit valide.

Logique et comportement

- Appel Claude avec le profil du lead, l'etape de sequence, l'historique des echanges precedents, et le ton de voix configure par le client.
- Garde-fou obligatoire au niveau du prompt systeme : aucune formulation ne doit suggerer qu'ARES remplace un commercial humain ; transparence sur la nature automatisee si la reglementation locale l'exige.
- Niveau d'autonomie configurable par workspace : supervision (validation humaine avant envoi) ou autonome (envoi direct).

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Etape de sequence prete a etre envoyee	Generation du message personnalise	Message en attente d'envoi, ou envoye directement selon autonomy_level	Sortie structuree : texte + canal cible
Mode supervision actif	Placer le message en validation humaine	Envoi bloque jusqu'a validation manuelle explicite	File de validation dediee dans le frontend

Etats et limites

- Echec ou timeout de generation : nouvelle tentative automatique limitee, puis alerte si echec persistant - jamais d'envoi d'un message vide ou generique par default.

Donnees et integrations

- Table messages ; lecture de agent_config_ares.autonomy_level.

4.9 Classification des reponses

Objectif

Comprendre automatiquement ce qu'un prospect repond, pour decider de la suite a donner sans intervention manuelle systematique.

Logique et comportement

- Classification en 5 categories fermees : Interesse / A recontacter plus tard / Pas interesse / Demande de retrait / Question hors-scope, avec extraction d'une date de relance si mentionnee.
- Toute classification a confiance faible est envoyee vers une file de traitement manuel plutot que de declencher une action automatique irreversible.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Reponse classée Interesse	Marquer le lead Qualifié pour transmission	Transfert vers le module Escalade (Section 4.10)	Priorité absolue
Reponse classée Demande de retrait	Ajout immédiat a suppression_list	Aucune sequence future sur ce contact, sur tous canaux	Portée workspace par défaut, portée globale si demande RGPD explicite
Reponse classée Pas	Arrêter la sequence	Lead archive avec motif	---

Declencheur	Decision du service FastAPI	Resultat	Note technique
interesse			
Classification ambigue (confiance faible)	Ne pas trancher automatiquement	File a traiter manuellement	Aucune decision irreversible sans confiance suffisante

Etats et limites

- Toute incertitude est traitee par default comme a valider manuellement, jamais par une action automatique risquee.

Donnees et integrations

- Tables lead_events, suppression_list.

4.10 Escalade et handoff commercial

Objectif

Transmettre un lead chaud a un humain (ou au CRM du client) au bon moment, sans perte de contexte.

Logique et comportement

- Le lead transmis emporte l'historique complet des echanges - aucune reprise a froid cote commercial.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Lead marque Interesse	Notifier l'equipe commerciale (in-app + option email) et synchroniser le CRM	Statut Transmis au commercial	Historique complet joint. Integrations : HubSpot, Salesforce, Zapier (+5000 apps)

Etats et limites

- CRM non connecte : le lead reste visible et actionnable dans VIRELYON, jamais bloque en attente d'une integration tierce.

Donnees et integrations

- Champ leads.statut_pipeline ; appel sortant vers CRM externe via n8n.

Endpoints FastAPI de reference

- POST /ares/{workspace_id}/leads/{lead_id}/escalate

4.11 Conformite et anti-abus

Objectif

Protéger la réputation d'envoi du client et respecter les obligations légales de prospection (RGPD zone Europe, réglementations locales, opt-out obligatoire).

Logique et comportement

- Toute communication contient un moyen de retrait clair et fonctionnel (lien de désinscription email, mention conforme aux exigences opérateur pour le SMS).
- Rate limiting strict par canal et par workspace, pour éviter le blacklisting d'un domaine d'envoi ou le dépassement de quota.
- Aucune donnée collectée hors ICP n'est conservée au-delà d'une durée définie et configurable par le client.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Contact ajoute a suppression_list	Propagation immediate a tous les canaux actifs pour ce contact	Blocage confirme sur l'ensemble des canaux	Le blocage a lieu au niveau du module Sequencage, avant l'appel au module Generation
Tentative d'envoi vers un	Blocage automatique avant	Aucun message n'est jamais genere	Le module Generation n'est

Declencheur	Decision du service FastAPI	Resultat	Note technique
contact deja en liste de suppression	generation de message	ni envoye	jamais invoque dans ce cas

Etats et limites

- Toute violation de quota ou de fenetre d'envoi est bloquee par construction, jamais detectee apres coup.

Donnees et integrations

- Table suppression_list ; journal d'audit conformite opposable en cas de controle.

4.12 Memoire et continuite (pgvector)

Objectif

Ne jamais faire repeter un client ou reprendre une conversation a zero apres une interruption technique ou une longue inactivite.

Logique et comportement

- Chaque interaction significative est indexee par embedding vectoriel (pgvector), permettant de retrouver le contexte pertinent meme apres plusieurs semaines d'inactivite sur un lead.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Reprise d'un echange apres interruption ou longue inactivite	Recherche semantique du contexte pertinent via pgvector	Message genere tenant compte de l'historique complet	Evite tout effet de reset percu comme non professionnel par le prospect

Etats et limites

- Aucun contexte disponible : generation standard sans reference a un historique inexistant, jamais une reference inventee.

Donnees et integrations

- Table embeddings ; lecture croisee de lead_events.

4.13 Multilinguisme

Objectif

Un meme ARES doit prospecter en francais, en anglais et, a terme, dans d'autres langues, sans reprise de code.

Logique et comportement

- Langue de generation determinee par default par la langue du workspace, avec possibilite de forcer la langue du message selon la langue detectee du prospect.
- Aucun texte de message n'est jamais code en dur dans une langue unique.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Generation d'un message	Lecture de workspace.locale, detection optionnelle de la langue du prospect	Message genere dans la langue appropriee	Detection de langue integree au module Classification

Etats et limites

- Langue non detectable avec confiance : repli sur la langue par default du workspace, jamais un melange de langues dans un meme message.

Donnees et integrations

- Champ workspace.locale.

4.14 Observabilite (lead_events)

Objectif

Garantir que chaque decision d'ARES est tracable - condition de credibilite produit et donnee source pour AURA.

Logique et comportement

- ARES ne calcule jamais lui-meme d'agregat de reporting : il produit des evenements structures, AURA les agrege.
- Chaque decision (qualification, scoring, envoi, classification, escalade) genere un evenement horodate et justifie, ecrit par app/events/.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Toute decision prise par un module 4.5 a 4.13, ou par le sourcing	Ecriture d'un evenement structure	Ligne ajoutee dans lead_events, consommable par AURA	Format stable et versionne, pense pour ne jamais casser les rapports AURA existants

Etats et limites

- Echec d'ecriture du journal : alerte technique immediate, traite comme critique - sans journal, ARES perd sa tracabilite.

Donnees et integrations

- Table lead_events (schema partage avec Big Data pour agregation AURA).

5. Agent APEX - Support client

EN UNE PHRASE

APEX est l'employe du support qui repond en moins de 3 secondes : il traite les questions recurrentes des clients finaux a partir de la base de connaissances fournie par l'entreprise, sur le canal de son choix, et transfere a un humain des que la situation devient sensible ou hors perimetre. Il ne remplace pas l'equipe support - il lui enleve les questions repetitives.

Toute la logique de decision d'APEX decrite ci-dessous est implementee dans le backend Python (FastAPI), module app/apex/ - jamais dans n8n (Section 2).

GARDE-FOU ANTI-HALLUCINATION - NON NEGOCIABLE

APEX ne repond jamais sur la base de sa connaissance generale. Chaque reponse doit etre fondee exclusivement sur les documents fournis par le client. Si aucun contenu pertinent n'est trouve, APEX escalade - il n'invente jamais. Implemente comme un controle explicite dans app/apex/service.py, jamais laisse a l'appréciation du prompt seul.

5.1 Les 6 reglages cote client

Reglage	Ce que le client saisit	Champ backend
1. Objectif principal	Une phrase libre (ex. Repondre aux questions produit et resoudre les demandes simples de facturation).	agent_config_apex.objectif_principal
2. Portee (scope autorise)	Ce qu'APEX gere vs. ce qu'il transfere direct a un humain (ex. gere : produit / statut de commande ; transfere : remboursements / litiges / resiliation).	agent_config_apex.portee (jsonb)
3. Formation (base de connaissances)	Upload FAQ, politique de remboursement, docs produit, anciens echanges support qui se sont bien passes (PDF, DOCX, TXT, saisie directe).	knowledge_base_documents + apex_knowledge_chunks (pgvector)
4. Niveau d'autonomie	Moyen (APEX repond aux recurrents, transfere le reste) ou Eleve (APEX gere tout sauf la liste d'exceptions definie en 2).	agent_config_apex.niveau_autonomie (enum)
5. Canaux	Cases a cocher : Email, WhatsApp, Chat web (widget/Typebot), Slack, integration SAV existant (ex. Live Person, Zendesk).	agent_config_apex.canaux_actifs (jsonb)
6. Ton de voix	Pro / Chaleureux / Direct / Creatif.	agent_config_apex.ton_de_voix (enum)

Test avant lancement : le client tape une question comme s'il etait un client reel, et voit exactement ce qu'APEX repondrait - avec sa base de connaissances deja chargee - avant de le brancher sur de vrais clients.

5.2 Ingestion et indexation de la base de connaissances

Objectif

Alimenter APEX avec les documents et FAQ du client et les rendre exploitables par recherche semantique - condition prealable a toute activation.

Logique et comportement

- Endpoint POST /apex/{workspace_id}/knowledge/upload : recoit le fichier, le stocke dans Supabase Storage, declenche l'indexation en tache de fond.
- Chaque document est decoupe en fragments (chunking) puis vectorise (pgvector) avant d'etre exploitable par le module RAG (Section 5.4).
- Un document modifie ou supprime declenche une purge puis une re-indexation - jamais un melange d'ancien et de nouveau contenu.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Upload d'un document	Decouper en fragments, vectoriser chaque fragment	Fragments inseres dans apex_knowledge_chunks, document marque indexe	Traitement asynchrone, jamais bloquant pour la requete HTTP d'upload
Echec d'indexation (format non supporte, fichier corrompu)	Isoler l'echec a ce fichier uniquement	Document marque echec + message explicite	Exception capturee et journalisee, ne bloque pas la file
Suppression / remplacement d'un document	Purger les fragments associes avant reindexation	Aucun fragment obsolete consultable	Transaction Postgres : purge et reindexation atomiques

Etats et limites

- Base de connaissances vide : agent en Configuration incomplete, ne peut pas passer Actif.
- Volume important de documents : traitement en file d'attente, jamais de degradation du reste du service.

Donnees et integrations

- Tables knowledge_base_documents, apex_knowledge_chunks.
- Supabase Storage pour les fichiers bruts.

Endpoints FastAPI de reference

- POST /apex/{workspace_id}/knowledge/upload
- DELETE /apex/{workspace_id}/knowledge/{document_id}

5.3 Reception multicanal de la demande

Objectif

Capter une demande entrante peu importe le canal, dans une structure de conversation unique et coherente.

Logique et comportement

- Chaque canal (chat web, email, WhatsApp, Slack) est relaye vers un endpoint FastAPI dedie : soit directement par le canal (webhook natif), soit via n8n comme simple relais transparent quand le canal ne supporte pas de webhook direct.
- Toute nouvelle demande cree ou rattache une conversation existante - jamais un nouveau fil pour chaque message d'un meme echange.
- Aucun canal n'est un prerequis technique : APEX doit fonctionner a 100% avec un seul canal actif.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Message reçu sur un canal actif	POST /apex/{workspace_id}/messages : creer ou rattacher la conversation	Conversation visible en statut En cours	Un endpoint generique, un connecteur n8n ou webhook natif par canal en amont
Canal en echec technique	Isoler l'echec a ce canal	Alerte + les autres canaux actifs continuent	Jamais de blocage total du service

Etats et limites

- Aucun canal connecte : agent en Configuration incomplete.

Donnees et integrations

- Table conversations.
- n8n en simple relais webhook pour les canaux sans integration API directe (WhatsApp Business API notamment).

Endpoints FastAPI de reference

- POST /apex/{workspace_id}/messages

5.4 Comprehension de la demande et RAG (recherche contextuelle)

Objectif

Comprendre l'intention du message et retrouver le contenu réellement pertinent dans la base de connaissances du client - sans jamais s'appuyer sur une connaissance hors base.

Logique et comportement

- Le service FastAPI appelle Claude pour classer l'intention en categories fermees : question_produit / reclamation / hors_scope / demande_humain, avec sortie structuree validee par un modele Pydantic.
- Recherche semantique pgvector sur apex_knowledge_chunks, strictement filtree par workspace_id au niveau de la requete SQL (jamais laissee au modele).
- Garde-fou non negociable : si aucun fragment ne depasse le seuil de pertinence configure, le service transmet directement au module Escalade (Section 5.6), sans appel de generation.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Nouveau message classe a traiter	Appel Claude - classification d'intention + recherche vectorielle	Intention + fragments pertinents avec score	Sortie validee par un schema Pydantic strict, rejet si non conforme
Aucun fragment au-dessus du seuil de pertinence	Ne jamais generer depuis la connaissance generale	Transfert direct vers le module Escalade	Controle fait en code Python, pas seulement dans le prompt
Intention classee demande_humain	Ignorer la recherche de contenu	Escalade immediate (Section 5.6)	Priorite absolue

Etats et limites

- Base de connaissances vide pour ce workspace : escalade systematique, motif explicite.
- Score ambigu : traite par default comme insuffisant - mieux vaut escalader a tort qu'halluciner.

Donnees et integrations

- Lecture apex_knowledge_chunks (isole par workspace_id).
- Ecriture conversation_events.

5.5 Generation de reponse

Objectif

Produire une reponse fondee exclusivement sur les fragments retrouves, dans le ton configure, sans jamais suggerer qu'APEX remplace un humain.

Logique et comportement

- Appel Claude avec : fragments pertinents, historique de conversation, ton de voix configure.
- Toute reponse generee est accompagnee d'une option explicite pour parler a un humain - jamais un cul-de-sac conversationnel.
- Reponse a confiance faible : jamais envoyee directement, transfert au module Escalade.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Fragments pertinents disponibles	Generation de la reponse, fondee uniquement sur ces fragments	Message insere, candidat au statut Resolu par l'agent	Sortie structuree : texte + option humain + score de confiance

Etats et limites

- Aucun message vide ou generique envoye par default.
- Ton de voix non configure : repli sur un ton neutre professionnel.

Donnees et integrations

- Table messages_apex ; lecture agent_config_apex.ton_de_voix.

5.6 Detection de confiance et escalade

Objectif

Decider automatiquement si un humain doit reprendre la main.

Logique et comportement

- Regles configurables par workspace (regles_escalade) : mots-cles sensibles, sentiment negatif, nombre d'echanges sans resolution, demande explicite d'un humain, sujet hors du perimetre de la base de connaissances.
- Toute ambiguite sur la confiance est traitee par default comme a escalader - jamais comme continuer a tenter de repondre.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Sentiment negatif detecte	Verifier le seuil configure	Escalade si seuil depasse, sinon poursuite avec vigilance	Score recalcule a chaque message
Nombre d'echanges sans resolution depasse	Comparer au plafond configure	Escalade automatique	Plafond configurable par workspace
Client demande explicitement un humain	Escalade immediate, sans condition	Transfert vers le module Handoff (Section 5.7)	Priorite absolue sur toute autre regle

Etats et limites

- Regles non configurees : valeurs par default prudentes appliquees, jamais aucune regle active.

Donnees et integrations

- Table regles_escalade ; ecriture conversation_events.

5.7 Handoff humain

Objectif

Transmettre une conversation a un humain sans perte de contexte.

Logique et comportement

- La conversation transmise emporte son historique complet.
- L'interruption de l'agent est strictement limitee au fil concerne - jamais l'agent entier mis en pause.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Escalade declenchee (Section 5.6)	Notifier l'equipe support, joindre l'historique complet	Conversation en statut Escalade	Notification via n8n (email/Slack) declenchee par un evenement FastAPI
Membre de l'equipe clique Reprendre la main	Basculer ce fil en mode humain	Reponses automatiques arretees sur ce fil	Les autres conversations en cours ne sont jamais affectees

Etats et limites

- Equipe non connectee a une notification : la conversation reste visible et actionnable dans VIRELYON.

Donnees et integrations

- Champ conversations.statut.

Endpoints FastAPI de reference

- PATCH /apex/{workspace_id}/conversations/{id}/status

5.8 Memoire et continuite de conversation

Objectif

Ne jamais faire repeter un client final, meme apres une interruption ou une reprise apres plusieurs jours.

Logique et comportement

- Chaque echange significatif est indexe par embedding vectoriel, permettant de retrouver le contexte pertinent apres une longue inactivite.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Reprise apres interruption prolongee	Recherche semantique du contexte pertinent via pgvector	Reponse tenant compte de l'historique complet	Meme index pgvector que le module RAG (Section 5.4)

Etats et limites

- Aucun contexte disponible : generation standard, jamais de reference inventee.

Donnees et integrations

- Embedding vectoriel des echanges ; lecture croisee conversation_events.

5.9 Multilinguisme

Objectif

Un meme APEX doit repondre en francais, en anglais et, a terme, dans d'autres langues, sans reprise de code.

Logique et comportement

- Langue de reponse determinee par la langue detectee du message entrant, avec repli sur workspace.locale.
- Aucun texte de reponse n'est jamais code en dur dans une langue unique - tous les prompts sont parametres par variable de langue.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Generation d'une reponse	Detection de la langue du message entrant	Reponse generee dans la langue detectee	Detection de langue en amont de l'appel Claude, dans le service FastAPI

Etats et limites

- Langue non detectable : repli sur workspace.locale, jamais de melange de langues.

Donnees et integrations

- Champ workspace.locale.

5.10 Satisfaction et feedback

Objectif

Mesurer la qualite percue du support pour alimenter AURA et l'amelioration continue.

Logique et comportement

- Collecte optionnelle d'une note a la cloture d'une conversation.
- Aucune relance insistante si le client final ne repond pas - respect de l'experience client.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Conversation cloturee	Proposer une note de satisfaction	Score enregistre si reponse	Une seule sollicitation, jamais de relance
Score de satisfaction bas	Generer un evenement dedie	Signal remonte a AURA pour analyse	Aucune action corrective automatique

Etats et limites

- Absence de reponse : aucune relance, la donnee reste simplement absente.

Donnees et integrations

- Table feedback_satisfaction ; ecriture conversation_events.

5.11 Observabilite (conversation_events)**Objectif**

Garantir que chaque decision d'APEX est tracable - condition de credibilite produit et donnee source pour AURA.

Logique et comportement

- APEX ne calcule jamais lui-meme d'agregat de reporting : il produit des evenements structures, AURA les agrege.
- Chaque decision genere un evenement horodate et justifie, ecrit par app/events/.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Toute decision prise par un module 5.2 a 5.10	Ecriture d'un evenement structure	Ligne ajoutee dans conversation_events, consommable par AURA	Ecriture systematique en fin de chaque service, jamais optionnelle

Etats et limites

- Echech d'ecriture du journal : alerte technique immediate, traite comme critique.

Donnees et integrations

- Table conversation_events.

6. Agent AURA - Reporting et Analytics

EN UNE PHRASE

AURA est l'analyste qui ne dort jamais : contrairement a ARES (qui va chercher sur internet) et APEX (qui apprend des documents fournis), AURA se branche directement sur les donnees reelles de l'entreprise - son CRM, sa facturation, ses tickets support - et consolide en continu avec la performance d'ARES et d'APEX. Il produit des rapports clairs sans jamais inventer un chiffre.

Toute la logique de decision d'AURA decrite ci-dessous est implementee dans le backend Python (FastAPI), module `app/aura/` - jamais dans `n8n` (Section 2).

GARDE-FOU ANTI-INVENTION DE DONNEES - NON NEGOCIABLE

AURA ne calcule et n'affiche jamais un chiffre qui n'est pas directement derive d'une donnee reellement synchronisee. Source non connectee, donnee absente, synchronisation perimee : AURA le signale explicitement plutot que d'estimer, d'arrondir de facon trompeuse ou d'inventer une valeur. Ce controle est un critere de recette, pas une preference de style.

6.1 Les 5 reglages cote client

Reglage	Ce que le client saisit	Champ backend (report_config)
1. Objectif principal	Une phrase libre (ex. Suivre l'evolution des ventes et du support client chaque semaine).	objectif_principal (text)
2. Sources de donnees a connecter	Bouton Connecter par outil : CRM (HubSpot / Salesforce), facturation (Stripe), tickets APEX (interne, sans action requise).	data_source_connections + declenchement via POST <code>/aura/{workspace_id}/connections/{type_source}</code>
3. Indicateurs a suivre	Cases a cocher parmi le catalogue : chiffre d'affaires, nouveaux clients, taux de reponse APEX, leads generes ARES, taux de desabonnement, delai moyen de resolution, etc.	indicateurs_actifs (jsonb, reference a <code>indicateurs_catalogue</code>)
4. Frequence et format	Ex. Rapport chaque lundi matin, envoye sur Slack + resume PDF.	frequence + jour_envoi + heure_envoi + format + canaux_diffusion
5. Ton de voix	Pro / Chaleureux / Direct / Creatif - meme logique qu'ARES et APEX.	ton_de_voix (enum)

Test avant lancement : le client peut demander a AURA montre-moi un exemple de rapport avec mes donnees actuelles avant d'activer les envois automatiques (Section 6.8).

6.2 Connexion et synchronisation des sources de donnees

Objectif

Brancher les outils reels du client et exposer, sans action du client, les donnees deja produites par ARES et APEX.

Logique et comportement

- Endpoint POST `/aura/{workspace_id}/connections/{type_source}` lance le flux d'autorisation puis la premiere synchronisation, en tache de fond.
- Sources internes (`lead_events`, `conversation_events`) toujours disponibles en lecture seule, sans action de connexion requise du client, des lors qu'ARES ou APEX est actif sur le meme workspace.
- Chaque synchronisation reussie met a jour `derniere_synchro`, base du garde-fou de fraicheur.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Client connecte une source externe (CRM, facturation)	Lancer l'autorisation puis la synchronisation	data_source_connections mis a jour, statut connecte	Un connecteur d'ingestion dedie par fournisseur
Echeance de synchronisation planifiee	Interroger la source connectee	Donnees a jour, derniere_synchro actualisee	Echec isole a cette source
ARES ou APEX actif sur le workspace	Exposer automatiquement les evenements internes	Indicateurs de performance disponibles sans action client	Lecture seule stricte

Etats et limites

- Aucune source externe connectee : AURA reste pleinement fonctionnel sur le perimetre performance_ares/performance_apex.

Donnees et integrations

- Table data_source_connections ; lecture lead_events, conversation_events.

Endpoints FastAPI de reference

- POST /aura/{workspace_id}/connections/{type_source}

6.3 Consolidation et calcul des indicateurs

Objectif

Calculer chaque indicateur actif a partir de donnees reellement synchronisees - jamais par estimation.

Logique et comportement

- Le calcul est execute en SQL depuis le service FastAPI (requetes SQLAlchemy) - jamais par Claude.
- Un indicateur n'est calcule que si sa source est connectee ET que la donnee existe pour la periode demandee.
- Le resultat de chaque calcul est conserve avec son statut (calcule / donnee_manquante / erreur), jamais uniquement la valeur finale.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Generation de rapport declenchee	Calculer chaque indicateur actif	Lignes inserees dans indicateurs_calcules, statut par indicateur	Traitement asynchrone, ne bloque jamais le dashboard
Source requise non connectee pour un indicateur	Ne pas calculer, ne pas substituer	statut_calcul = donnee_manquante	Transmis directement au module Signalement (Section 6.4)

Etats et limites

- Signaux insuffisants : jamais une valeur approximative - statut donnee_manquante explicite.

Donnees et integrations

- Table indicateurs_calcules.

6.4 Signalement des donnees manquantes (anti-invention)

Objectif

Garantir qu'aucun chiffre n'est jamais invente, estime ou extrapole silencieusement.

Logique et comportement

- Tout indicateur en statut donnee_manquante ou erreur est explicitement signale dans le rapport final, jamais masque ni remplace par une valeur neutre (zero, tiret, moyenne estimee).

- Le motif est toujours explicite et comprehensible par le client (ex. source de facturation non connectee, et non un code technique brut).

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Indicateur en statut donnee_manquante ou erreur	Ne jamais tenter de compenser par une estimation	Entree creee dans donnees_manquantes_signalees	Controle fait en code Python avant tout appel a Claude pour la narration

Etats et limites

- Aucune tolerance : pas de valeur par default acceptable.

Donnees et integrations

- Table donnees_manquantes_signalees.

6.5 Generation du rapport narratif

Objectif

Transformer des indicateurs deja calcules en texte clair, sans jamais laisser le modele recalculer ou halluciner un chiffre.

Logique et comportement

- Appel Claude avec en entree les valeurs deja figees d'indicateurs_calcules, l'objectif, le ton de voix, et la liste des donnees manquantes.
- Garde-fou au niveau du prompt systeme : aucune reformulation d'une donnee manquante en valeur approximative.
- Le texte genere structure toujours un resume en tete, puis le detail par indicateur, puis une section dediee aux donnees manquantes le cas echeant.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Indicateurs calcules disponibles pour la periode	Generation du rapport a partir des valeurs figees uniquement	Texte insere dans rapports_generes, statut brouillon ou apercu	Sortie structuree : resume + detail + section donnees manquantes

Etats et limites

- Aucun rapport vide ou generique n'est jamais diffuse par default - mieux vaut retarder l'envoi et alerter.

Donnees et integrations

- Table rapports_generes.

6.6 Diffusion multicanal

Objectif

Livrer le rapport selon la frequence, le format et le(s) canal(aux) choisis, sans action du client.

Logique et comportement

- Declenchement planifie selon report_config.
- Le canal export_webhook pousse les memes donnees que les autres canaux vers l'URL fournie par le client (Power BI, Google Data Studio, Notion, Google Sheets) - un seul connecteur generique, pas un connecteur dedie par outil de BI.
- Le dashboard interne reste toujours disponible par default, meme si aucun canal externe n'est connecte.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Echeance de diffusion	Envoyer le rapport sur chaque canal	rapports_generes.statut passe a	Diffusion Slack/email relayee

Declencheur	Decision du service FastAPI	Resultat	Note technique
planifiee atteinte	actif	diffuse	par n8n ; export_webhook et dashboard servis directement par FastAPI
Echec d'envoi sur un canal	Isoler l'echec a ce canal	Alerte + rapport reste consultable sur le dashboard	Jamais un rapport totalement perdu pour le client

Etats et limites

- Canal non connecte au moment de la diffusion : ignore avec journalisation, jamais un blocage des autres canaux.

Donnees et integrations

- Champs rapports_generes.statut et canaux_diffuses.

6.7 Bulletin de performance des agents (ARES + APEX)

Objectif

Consolider, en langage clair, la performance d'ARES et d'APEX sur la periode - en lecture seule stricte.

Logique et comportement

- Lecture agregee de lead_events (ARES) et conversation_events (APEX), isolee par workspace_id.
- Un zero chiffre (agent actif, rien produit sur la periode) est une vraie donnee ; un agent inactif produit un signalement distinct, jamais un zero trompeur.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Rapport avec indicateurs performance_ares ou performance_apex actifs	Agreger les evenements sur la periode, en lecture seule	Valeurs inserees dans indicateurs_calcules, categorie performance_ares / performance_apex	Aucune ecriture d'AURA dans les tables sources d'ARES ou d'APEX
ARES ou APEX inactif ou non configure	Ne pas produire de valeur substituee	Indicateur correspondant signale comme non disponible	Distinction stricte entre zero activite reelle et agent non actif

Etats et limites

- Un agent inactif sur le workspace ne doit jamais apparaitre avec une performance a zero sans explication.

Donnees et integrations

- Lecture lead_events, conversation_events ; ecriture indicateurs_calcules.

6.8 Mode apercu (test avant activation)

Objectif

Permettre au client de valider le rendu d'un rapport avant d'activer les envois automatiques.

Logique et comportement

- Le client peut demander un apercu a tout moment, y compris avant l'activation complete.
- Un apercu n'est jamais compte comme une diffusion et ne declenche aucun envoi automatique.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Client demande un apercu	GET /aura/{workspace_id}/reports/previe w	Rapport affiche, non diffuse automatiquement	Peut etre demande un nombre illimite de fois

Declencheur	Decision du service FastAPI	Resultat	Note technique
Aucune source externe connectee au moment de l'aperçu	Generer l'aperçu sur le seul perimetre disponible (performance ARES/APEX)	Rapport limite + signalement explicite des indicateurs business manquants	Le client voit concretement l'effet d'une non-connexion avant de s'engager

Etats et limites

- Un aperçu genere n'ecrase jamais un rapport deja diffuse et n'entre jamais dans l'historique de diffusion.

Donnees et integrations

- Table rapports_generes (statut = aperçu).

Endpoints FastAPI de reference

- GET /aura/{workspace_id}/reports/preview

6.9 Personnalisation avancee

Objectif

Permettre l'ajout d'indicateurs hors catalogue standard, sans bloquer le fonctionnement standard.

Logique et comportement

- Toute demande hors catalogue est enregistree comme demande de configuration additionnelle.
- Necessite une intervention technique dediee, hors auto-service standard.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Client demande un indicateur hors catalogue	Enregistrer la demande, ne pas tenter une implementation automatique	Demande transmise pour configuration technique	Hors flux applicatif automatise - ticket de configuration

Etats et limites

- Demande non traitee : n'affecte jamais les indicateurs standards deja actifs.

Donnees et integrations

- Hors modele de donnees standard.

6.10 Multilinguisme

Objectif

Un meme AURA doit produire ses rapports en francais, en anglais et, a terme, dans d'autres langues, sans reprise de code.

Logique et comportement

- Langue de generation determinee par workspace.locale.
- Aucun texte de rapport n'est jamais code en dur dans une langue unique.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Generation d'un rapport	Lecture de workspace.locale	Rapport genere dans la langue du workspace	Coherent avec le multilinguisme d'APEX (Section 5.9)

Etats et limites

- Langue non detectable : repli sur la langue par defaut du workspace.

Donnees et integrations

- Champ workspace.locale.

6.11 Observabilite (aura_events)

Objectif

Garantir que chaque decision d'AURA est tracable - condition de credibilite produit.

Logique et comportement

- Chaque decision des modules 6.2 a 6.10 ecrit un evenement horodate et justifie dans aura_events.
- Les evenements d'AURA ne sont consommes par aucun autre agent - mais restent indispensables a l'audit interne.

Declencheur - Decision - Resultat

Declencheur	Decision du service FastAPI	Resultat	Note technique
Toute decision prise par un module 6.2 a 6.10	Ecriture d'un evenement structure	Ligne ajoutee dans aura_events	Format stable et versionne

Etats et limites

- Echec d'ecriture du journal : alerte technique immediate, traite comme critique.

Donnees et integrations

- Table aura_events.

7. Prompts systeme (contrat entree/sortie)

Chaque prompt est un fichier versionne dans app/ares/prompts.py, app/apex/prompts.py ou app/aura/prompts.py, associe a un schema Pydantic de sortie. Le texte definitif (formulation, exemples) reste ajustable ; le contrat d'entree/sortie ci-dessous est fige des maintenant pour permettre le developpement en parallele des trois modules.

7.1 Prompts ARES

- Qualification ICP (Section 4.5) - entrees : workspace_icp_config (criteres actifs), profil du lead brut. Sortie Pydantic : qualifie (booleen), confiance (0-1), motif (texte court).
- Generation de message (Section 4.8) - entrees : profil du lead, etape de sequence, ton de voix configure, historique des echanges. Regle non negociable : jamais suggerer un remplacement humain. Sortie : texte du message + canal cible.
- Classification de reponse (Section 4.9) - sortie Pydantic : categorie fermee (parmi les 5 definies), confiance (0-1), date de relance extraite si presente.
- Decision de prochaine action (transverse) - sortie Pydantic : action (parmi le catalogue ferme) + justification courte pour tracabilite. Entree additionnelle : palier de score du lead et compteur de relances effectuees.

7.2 Prompts APEX

- Classification d'intention et recherche contextuelle (Section 5.4) - entrees : message, historique, fragments candidats. Sortie Pydantic : intention (enum fermee), fragments_pertinents (liste + score), confiance (0-1).
- Generation de reponse (Section 5.5) - entrees : fragments pertinents, historique, ton de voix. Regles non negociables : jamais suggerer un remplacement humain ; jamais repondre hors base fournie. Sortie : texte + indicateur de confiance.
- Detection de sentiment et decision d'escalade (Section 5.6) - sortie Pydantic : sentiment (echelle), declencheurs actifs identifies, decision (continuer / escalade), justification courte.

7.3 Prompts AURA

- Generation narrative (Section 6.5) - entrees : valeurs figees d'indicateurs_calcules, objectif, ton de voix, donnees manquantes. Regle non negociable : jamais produire, arrondir ou deduire un chiffre non fourni explicitement en entree.
- Bulletin de performance des agents (Section 6.7) - entrees : agregats deja calcules de lead_events et conversation_events, objectif, ton de voix. Sortie : texte clair, sans jargon technique.
- Detection de disponibilite d'un indicateur (Section 6.3 / 6.4) - sortie Pydantic : disponible (booleen), motif si indisponible (categorie fermee).

7.4 Regle transverse

VALIDATION PYDANTIC STRICTE

Toute sortie de Claude consommee par le backend passe par un schema Pydantic strict avec validation a la reception. Une sortie non conforme au schema est traitee comme un echec technique (retry, puis escalade si persistant) - jamais interpretee de facon permissive.

8. Integrations tierces (par agent)

Chaque integration ci-dessous est orchestree depuis le backend Python (FastAPI) - directement via un client HTTP, ou via n8n en simple relais quand l'outil tiers ne propose pas d'API directe (Section 2).

Integration	Agent	Role	Bloquant ?
Google Maps API	ARES	Sourcing d'entreprises geolocalisees.	Non

Integration	Agent	Role	Bloquant ?
Hunter API	ARES	Verification et recherche d'emails professionnels.	Non
Apify API	ARES	Scraping structure multi-sources.	Non
LinkedIn (public)	ARES	Sourcing de decideurs et engagement complementaire. Acces a valider juridiquement selon le pays de la source ; ARES prevoit un mode degrade sans LinkedIn.	Non
Enrichissement de donnees B2B (optionnel)	ARES	Complete les profils de leads bruts pour ameliorer le scoring (Section 4.6). Ameliore la qualite du score sans etre requis pour qualifier un lead.	Non
Scraping web generique	ARES	Enrichissement de profil avant qualification (donnees publiques uniquement).	Non
Email (SMTP / API)	ARES + APEX	Canal de communication principal.	Recommande
WhatsApp Business API	ARES + APEX	Canal conversationnel additionnel.	Non
SMS (Orange Developer API)	ARES	Canal additionnel, jamais prerequis.	Non
Slack	ARES + APEX + AURA	Canal conversationnel client (ARES/APEX) ou canal de diffusion (AURA).	Non
Widget de chat web (Typebot)	APEX	Canal principal universel.	Non
CRM (HubSpot, Salesforce)	ARES + APEX + AURA	ARES : transmission des leads. APEX : contexte client, creation de ticket. AURA : indicateurs business.	Non
Helpdesk (Zendesk, Freshdesk)	APEX	Enrichit le contexte, creation de ticket lors d'escalade.	Non
Bases de connaissances externes (Notion, Google Drive, Confluence)	APEX	Source d'indexation alternative a l'upload manuel.	Non
Facturation (Stripe)	AURA	Alimente les indicateurs de revenu.	Non
Zapier (+5000 apps)	Tous	Extension d'ecosysteme via connecteurs no-code.	Non
Reseau de support live Orange	APEX	Canal d'escalade additionnel pour clients onboardees via Orange.	Non
Export webhook (BI externe : Power BI, Google Data Studio, Notion)	AURA	Canal generique de diffusion vers l'outil BI deja utilise par le client.	Non
lead_events (interne)	AURA	Alimente le bulletin de performance ARES.	Non - toujours disponible
conversation_events (interne)	AURA	Alimente le bulletin de performance APEX.	Non - toujours disponible
Stockage + indexation vectorielle (Supabase Storage / pgvector)	APEX	Alimente la base de connaissances.	Oui - seul bloquant reel

Principe transverse aux trois agents : aucune integration externe n'est un prerequis bloquant a l'activation, a l'exception du stockage/indexation vectorielle pour APEX - sans base de connaissances, l'agent n'a rien a repondre.

9. Parcours transversaux inter-agents

9.1 Cycle de vie complet d'un lead (ARES)

Sourcing / import (Section 4.4) -> leads_raw -> Qualification ICP (Section 4.5) -> Scoring et priorisation (Section 4.6, avec paliers de relance) -> Sequencage multicanal (Section 4.7) - Generation de messages (Section 4.8) -> reponse du prospect -> Classification (Section 4.9) -> selon le cas : Escalade commerciale (Section 4.10), archivage, ou ajout a suppression_list (Section 4.11) -> a chaque etape, ecriture d'un evenement (Section 4.14) consomme ensuite par AURA.

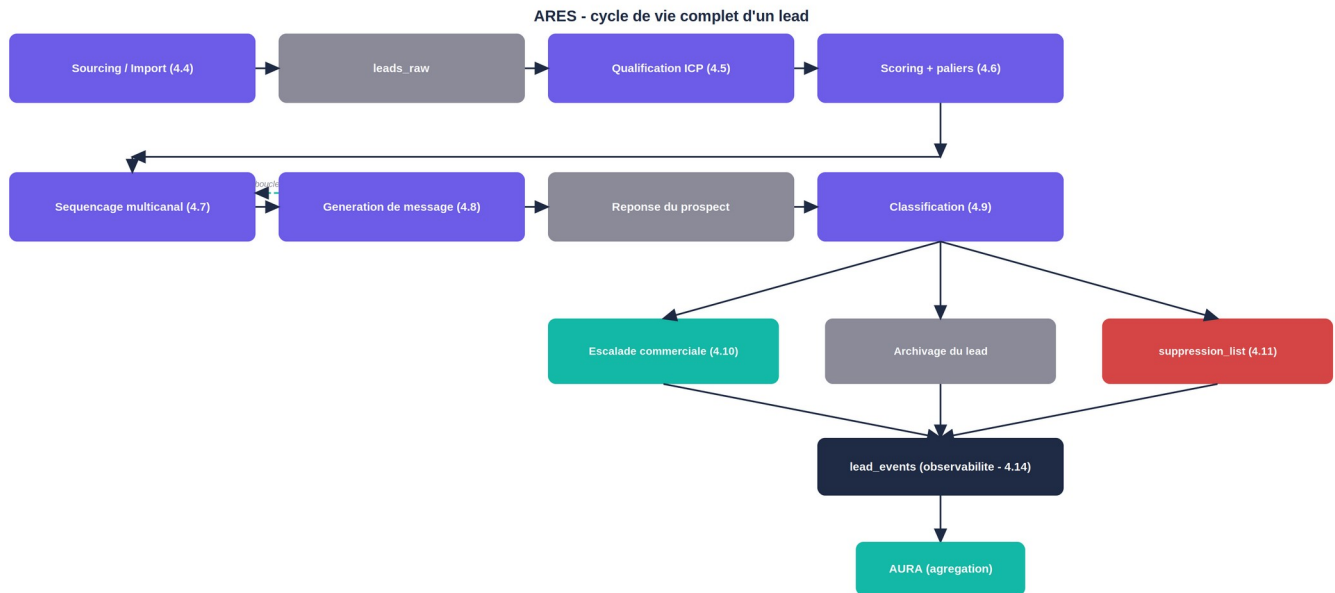


Figure 2 - cycle de vie complet d'un lead ARES, du sourcing a l'agregation AURA.

9.2 Cycle de vie complet d'une conversation (APEX)

Reception multicanal (Section 5.3) -> Comprehension et RAG (Section 5.4) -> Generation de reponse (Section 5.5) - Detection de confiance et escalade (Section 5.6) -> reponse envoyee ou Handoff humain (Section 5.7) -> Cloture -> Feedback (Section 5.10) -> evenement ecrit a chaque etape (Section 5.11).

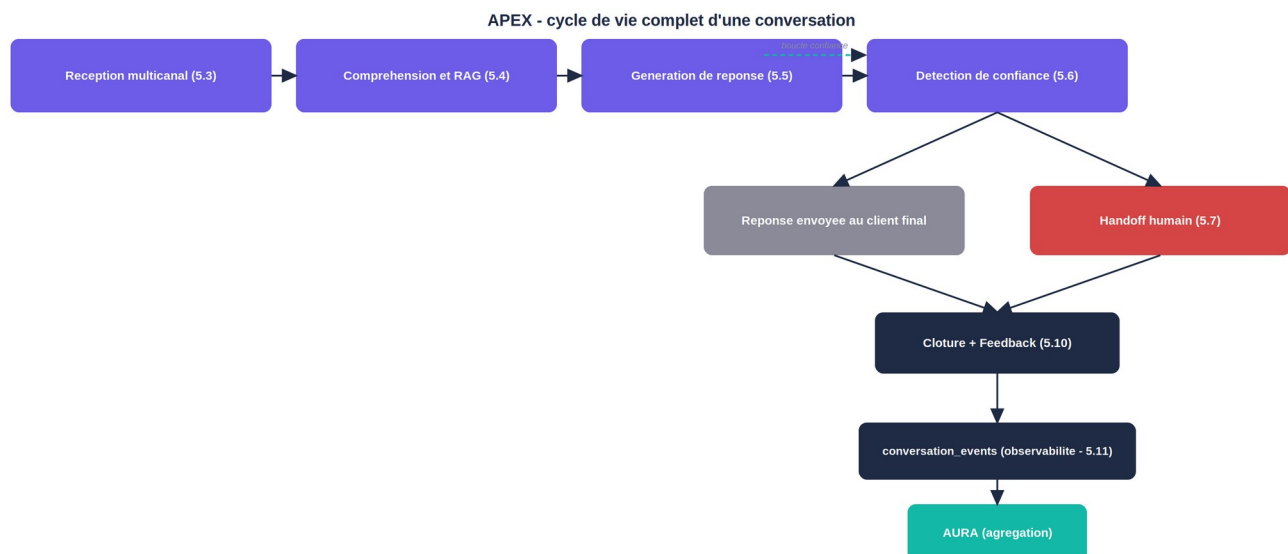


Figure 3 - cycle de vie complet d'une conversation APEX, jusqu'a l'agregation AURA.

9.3 Cycle de vie complet d'un rapport (AURA)

Configuration initiale (Section 6.1) -> Connexion des sources (Section 6.2) -> declenchement planifie ou apercu (Section 6.8) -> Consolidation et calcul (Section 6.3) -> Signalement des donnees manquantes (Section 6.4) -> Generation narrative (Section 6.5) -> Diffusion multicanal (Section 6.6) -> evenement ecrit a chaque etape (Section 6.11).

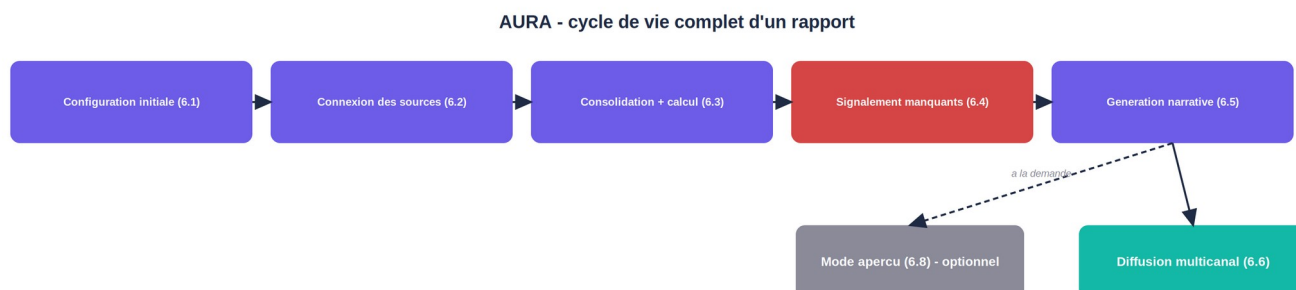


Figure 4 - cycle de vie complet d'un rapport AURA, avec le mode apercu en branche optionnelle.

9.4 Flux ARES / APEX vers AURA (lecture seule stricte)

AURA est le seul agent qui lit les evenements produits par les deux autres. Cette lecture est strictement en lecture seule, isolee par workspace_id, et ne modifie jamais les donnees sources. Garanti par les permissions Postgres au niveau du role applicatif utilise par le module aura/ du backend : ce role n'a aucun droit d'ecriture sur les tables lead_events et conversation_events.

9.5 Boucles d'apprentissage (hors perimetre immediat)

Un retour manuel de l'equipe (ce lead n'etait pas qualifie, cette reponse etait fausse, ce rapport n'etait pas ce que j'attendais) doit pouvoir etre enregistre des la premiere version dans lead_events, conversation_events ou aura_events. L'exploitation automatique pour ameliorer le scoring, la base de connaissances ou le catalogue d'indicateurs reste hors perimetre immediat. Ne pas fermer cette porte dans le modele de donnees evite une migration couteuse plus tard.

10. Garde-fous non negociables (consolides)

10.1 Communs aux trois agents

- Positionnement : aucun texte genere par ARES, APEX ou AURA ne doit suggerer qu'il remplace un humain. Augmenter, jamais remplacer.
- Toute incertitude (qualification, classification, confiance, disponibilite d'un indicateur) est traitee par default comme a valider manuellement, jamais par une action automatique irreversible.
- Chaque decision et chaque action est journalisee : aucune action silencieuse, sur aucun des trois agents.
- Isolation stricte des donnees par workspace (Row Level Security) sur toutes les tables des trois agents, verifiee cote base ET cote backend FastAPI (double controle).
- Aucune integration externe n'est un prerequis bloquant, a la seule exception de l'indexation vectorielle pour APEX.
- Claude ne calcule jamais un agregat numerique : tout calcul passe par du SQL execute depuis le service FastAPI.
- Toute sortie de Claude passe par un schema Pydantic strict avec validation a la reception.

10.2 Specifiques a ARES

- ARES ne choisit jamais une action hors du catalogue ferme (Section 4.3).

- Aucune plage d'effectif fixe n'est appliquee comme filtre ICP : la cible reste toute entreprise de service B2B, sans fourchette de taille.
- Le scraping et l'enrichissement de profil ne portent jamais sur des donnees privees - uniquement des informations publiques.
- Tout lead est deduplique avant toute ecriture en base, sans exception.
- Le module de qualification est strictement en lecture seule sur la configuration ICP.
- Le canal SMS Orange est un canal additionnel - jamais un prerequisite.

10.3 Specifiques a APEX

- APEX ne repond jamais sur la base de sa connaissance generale - uniquement sur les documents fournis par le client.
- Aucun message vide ou generique n'est jamais envoye par default - mieux vaut escalader.
- APEX respecte strictement la portee (scope) definie par le client (reglage 2) : tout sujet hors portee declenche une escalade immediate.

10.4 Specifiques a AURA

- AURA ne calcule et n'affiche jamais un chiffre qui n'est pas directement derive d'une donnee reellement synchronisee.
- Un indicateur non calculable est toujours signale explicitement, jamais masque ni remplace par une valeur neutre.
- AURA ne presente jamais une donnee perimee comme si elle etait a jour : un seuil de fraicheur de synchronisation est configurable par source.
- AURA n'ecrit jamais dans les tables sources d'ARES ou d'APEX - lecture seule stricte.

11. Exigences non-fonctionnelles

11.1 Performance

- Qualification et scoring (ARES), indexation et generation narrative (APEX/AURA) traitees de facon asynchrone (BackgroundTasks FastAPI ou file d'attente dediee).
- Cible : endpoints de lecture sous 2 secondes.
- Actualisation des conversations APEX en cours via Supabase Realtime, sans rechargement.

11.2 Securite et conformite

- Isolation stricte par workspace (Row Level Security Supabase), double controle base + backend.
- Aucun secret d'integration (cle API CRM, cle Stripe, cle Claude, cles ARES) expose cote client - stocke en variables d'environnement du service FastAPI.
- suppression_list invulnérable : un blocage reste actif meme apres changement de configuration ICP ou de sequence.
- Droit a l'oubli sur demande explicite, coherent pour les trois agents.
- Historique de conversation soumis aux memes regles RGPD que les autres donnees clients.

11.3 Scalabilite

- File d'attente pour absorber : les pics d'ingestion (sourcing en masse ARES, import CSV volumineux), les pics de generation (rapports AURA programmes au meme horaire), les pics de conversations simultanees (APEX).
- Pas de degradation de l'experience des autres workspaces en cas de pic sur un workspace.

11.4 Multilinguisme

Aucune reprise de code requise pour ajouter une langue. Tous les prompts sont parametres par variable de langue, et la langue de reponse est determinee par workspace.locale ou par detection du message entrant. Condition directe de l'objectif international du produit.

11.5 Rate limiting

- Table de quotas configurable par canal et par workspace (agent_config.quotas_par_canal), valeurs par default prudentes.
- Table de paliers de relance configurable par score et par workspace (agent_config_ares.paliers_relance_score, Section 4.6.1).
- Cote AURA : seuil de fraicheur de synchronisation configurable par source, pour eviter d'afficher une donnee perimee comme si elle etait a jour.

11.6 Observabilite

Chaque decision d'agent est journalisee (lead_events, conversation_events, aura_events). Un echec d'ecriture du journal est traite comme critique, jamais silencieux - sans journal, l'agent perd sa tracabilite et donc sa credibilite devant un client B2B exigeant.

12. Feuille de route technique

12.1 Statut au moment de la redaction

- Acte - specification fonctionnelle complete d'ARES, APEX et AURA, garde-fous inclus, dans ce document unique.
- Acte - architecture unique : un backend Python FastAPI porte toute la logique metier des trois agents ; n8n recentre sur les connecteurs externes uniquement.
- Non commence - aucun code du backend FastAPI n'existe a ce jour pour les trois agents reunis. Ce document est le point de depart, pas un etat des lieux d'un existant.

12.2 Chantier technique immediat (dans l'ordre)

- 1. Bootstrap du projet FastAPI selon l'arborescence (Section 2.4), connexion Supabase Postgres (SQLAlchemy async), premieres migrations Alembic pour les tables (Section 3).
- 2. Client Claude API unique et partage (module app/claude/), avec gestion des retries et validation systematique des sorties structurees par Pydantic.
- 3. Module ARES : sourcing/ingestion (Section 4.4) puis Qualification ICP (Section 4.5) - sans lead qualifie, aucun autre module n'est testable.
- 4. Module APEX : ingestion base de connaissances (Section 5.2) puis reception multicanal (Section 5.3) - sans base indexee, aucun autre module n'est testable.
- 5. Module AURA : configuration (Section 6.1) puis bulletin de performance des agents (Section 6.7), qui ne depend d'aucune integration externe - le module demonstrable le plus rapidement en interne.
- 6. Garde-fous en premier, pas en dernier : catalogue ferme d'ARES (Section 4.3), anti-hallucination d'APEX (Section 5.4) et anti-invention d'AURA (Section 6.4) doivent etre ecrits et testes avant la generation elle-meme.
- 7. Observabilite (lead_events, conversation_events, aura_events) branchee des le premier endpoint fonctionnel de chaque agent, pas en fin de sprint.

12.3 Points ouverts a valider avant implementation complete

- Choix definitif du mecanisme de taches de fond (BackgroundTasks FastAPI simple vs. file d'attente dediee Celery / Redis / RQ) - a trancher selon le volume reel observe en test, pas par anticipation.
- Texte definitif des prompts systeme (Section 7) - le contrat d'entree/sortie est fige, la formulation reste a valider avant mise en production.

- Schema exact des champs de lead_events et conversation_events exploitables pour le bulletin de performance AURA (Section 6.7) - a confirmer lors de la redaction des migrations Alembic.
- Modele Claude par point d'appel (Haiku, Sonnet, Opus) - a arbitrer selon les tests de qualite et le budget d'inference, jamais fige en dur dans le code.

13. Glossaire

- workspace_id - identifiant unique du client, utilise pour l'isolation stricte des donnees (Row Level Security) sur les trois agents.
- ICP (Ideal Customer Profile) - profil client cible defini par le client (secteurs, zones, mots-cles, effectif optionnel).
- RAG (Retrieval Augmented Generation) - technique consistant a fournir a Claude des fragments issus d'une recherche vectorielle avant de generer une reponse (utilisee par APEX, Section 5.4).
- Statuts du pipeline lead (ARES) - Identifie -> Contacte -> Repondu -> Qualifie -> Transmis au commercial.
- qualification_status (ARES) - qualifie / rejete / a_valider.
- statut_envoi (message ARES) - en_attente / envoye / echec / bloque_suppression_list.
- autonomy_level (ARES / APEX) - ARES : supervision / autonome. APEX : Moyen / Eleve.
- Conversation APEX - En cours -> Resolu par l'agent -> Escalade.
- intention (APEX classification) - question_produit / reclamation / hors_scope / demande_humain.
- Rapport AURA - brouillon -> apercu -> diffuse (ou echec_diffusion).
- Statut de calcul d'indicateur (AURA) - calcule / donnee_manquante / erreur.
- Motif de donnee manquante (AURA) - source_non_connectee / donnee_absente_periode / synchro_perimee / erreur_source.
- n8n - outil d'automatisation no-code, utilise dans VIRELYON exclusivement comme connecteur vers des outils tiers, jamais comme porteur de decision.
- pgvector - extension Postgres pour la recherche vectorielle (memoire d'ARES et RAG d'APEX).
- Pydantic - librairie Python de validation de schemas, utilisee pour toutes les sorties structurees de Claude.

VIRELYON - AI WORKFORCE PLATFORM

Fin du document - CDCF Unifie v3.0